

DUOC UC - ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES

Especificación de Requisitos de Software

Proyecto: TeqRuta

Fecha: 29/Sep/2026

Especificación de Requisitos según estándar de IEEE 830.

Ficha del documento

Fecha	Revisión	Autor	Modificación
20/09/2026	1.0	Hans Morales, Martina Norambuena, Williams Contreras.	Creación inicial del documento ERS basado en el caso TEQMED.
22/09/2026	1.1	Hans Morales	Ajuste inicial de alcances y restricciones offline.
23/09/2026	1.2	Martina Norambuena	Inclusión de detalles sobre la captura de odometría y peajes.
25/09/2026	1.3	Williams Contreras	Definición técnica de la API REST y arquitectura Flask.
27/09/2026	1.4	Hans Morales	Actualización de requerimientos de seguridad con tokens JWT.
29/09/2026	1.5	Martina Norambuena	Revisión final y validación de flujos para gerencia y finanzas.

Documento validado por las partes en fecha: 29/09/2026

Por el cliente (TEQMED SpA)	Por la empresa suministradora
[Firma]	[Firma]
Sr./Sra. Director de Operaciones	Sr./Sra. Líder de Proyecto / Desarrollador

1. Introducción

1.1. Propósito

El propósito de este documento es definir detalladamente los requisitos funcionales y no funcionales del sistema "TeqRuta". Este documento está dirigido al equipo de desarrollo de software, diseñadores de interfaces y a la gerencia de TEQMED SpA, sirviendo como contrato técnico y guía para la construcción, pruebas y validación del producto.

1.2. Ámbito del Sistema

- Nombre del sistema: TeqRuta.
- Lo que el sistema hará: Reemplazará las libretas de registro manuales de los técnicos en terreno por una aplicación móvil capaz de registrar traslados, odometría y peajes sin necesidad de conexión a internet. El sistema validará estas distancias, calculará el costo exacto basado en el precio dinámico del combustible, y proveerá una plataforma web para que la jefatura apruebe gastos y gerencia analice la rentabilidad de cada contrato médico.
- Lo que el sistema no hará: En esta primera fase, no se integrará de forma directa o automática con el ERP contable actual de la empresa para la facturación.
- Beneficios y metas: Eliminar la pérdida de información en papel, evitar los sobrecostos por diferencias entre los kilómetros declarados y los reales, y visibilizar de forma exacta cuánto cuesta operativamente atender a cada clínica (rentabilidad por cliente).

1.3. Definiciones, Acrónimos y Abreviaturas

- ERS: Especificación de Requerimientos de Software.
- Odometría: Medición de la distancia recorrida por un vehículo, clave para el cálculo de costos en el sistema.
- Offline-first: Capacidad crítica de la aplicación móvil de funcionar con total normalidad guardando datos en la memoria del teléfono cuando no hay señal de internet.
- UUID: Identificador Único Universal (código alfanumérico) usado para que un viaje no se guarde dos veces por error.
- JWT: JSON Web Token, estándar utilizado para mantener la sesión segura del usuario.

1.4. Referencias

- Documento de Caso: Caso_DSY1105_TeqRuta.pdf (Contexto de TEQMED SpA).
- Estándar IEEE 830-1998 para Especificaciones de Requisitos de Software.

1.5. Visión General del Documento

Este documento consta de una descripción general del producto y su contexto operativo en la Sección 2. Posteriormente, en la Sección 3, se detallan exhaustivamente los requerimientos de interfaz, seguidos por el listado de requerimientos funcionales (acciones del sistema) y no funcionales (rendimiento, seguridad, disponibilidad).

2. Descripción General

2.1. Perspectiva del Producto

TeqRuta es una plataforma independiente (standalone) compuesta por dos componentes principales: una aplicación móvil desarrollada en Android Nativo (Kotlin) y un backend centralizado expuesto mediante una API REST (Flask/Python). El sistema se comunicará unidireccionalmente con una API pública externa para extraer el precio diario del combustible, vital para los cálculos de costos de la empresa.

2.2. Funciones del Producto

- Gestión y registro de viajes (inicio/fin) con captura de odómetro.

- Captura fotográfica de comprobantes de peaje con compresión de imagen.
- Sincronización encolada de datos (manejo de modo offline).
- Motor de validación de desviaciones de kilometraje (declarado vs oficial).
- Generación de métricas de rentabilidad y exportación de datos a CSV.
- Mantenedores (ABM) de flota vehicular, clínicas destino y tarifas paramétricas.

2.3. Características de los Usuarios

- Técnico de Servicio: Usuario móvil operativo que viaja por la región. Nivel educacional técnico, experiencia básica en uso de smartphones. Requiere flujos muy rápidos y pantallas con alto contraste para operar en terreno.
- Jefe de Servicio Técnico: Usuario de oficina. Nivel medio en uso de PC. Su foco es la supervisión, por lo que necesita interfaces de validación rápidas (gestos de aprobación/rechazo).
- Gerencia / Finanzas: Usuario administrativo y analítico. Experiencia avanzada en Excel. Foco en la lectura de gráficos, reportes de costos y descarga de bases de datos.
- Administrador: Usuario con conocimientos técnicos avanzados. Responsable de mantener actualizadas las variables del sistema (patentes, rendimiento de vehículos, etc.).

2.4. Restricciones

- Limitaciones de hardware/software: La aplicación móvil está restringida exclusivamente a dispositivos con sistema operativo Android.
- Operaciones paralelas: La app debe soportar el trabajo sin conexión por hasta 8 horas continuas.
- Legal: Se debe cumplir la ley de protección de datos personales en el manejo de credenciales y rutas.
- Equipo: El desarrollo debe completarse considerando la capacidad de un único desarrollador Full-Stack (400 horas estimadas).

2.5. Suposiciones y Dependencias

- Se asume que los vehículos de la flota (o particulares asignados) cuentan con odómetros funcionales y legibles.
- Se asume que los técnicos cuentan con planes de datos móviles, aunque la cobertura sea intermitente en las rutas a Chillán o Los Ángeles.
- El cálculo automático depende de la estabilidad de la API pública de combustibles.

2.6. Requisitos Futuros

- Integración vía Web Service o API nativa con el ERP contable de la empresa para automatizar el pago de viáticos y facturación a clínicas.

3. Requisitos Específicos

3.1 Requisitos comunes de las interfaces

3.1.1 Interfaces de usuario

- Móvil (Técnicos): Desarrollada en Jetpack Compose. Contará con un menú de navegación inferior (Bottom Navigation). Las pantallas priorizarán inputs numéricos grandes para los odómetros y botones de acción flotantes (FAB) visibles a la luz del día.
- Web (Jefatura/Gerencia/Admin): Interfaces HTML/CSS accesibles desde navegador de escritorio, con un panel lateral (Sidebar) para navegación y áreas de contenido basadas en tablas de datos (DataTables) y tarjetas de resumen (Cards).

3.1.2 Interfaces de hardware

- El sistema requiere permisos de acceso a la cámara del dispositivo Android para la captura óptica de los comprobantes de peaje físicos.

3.1.3 Interfaces de software

- Motor de Base de Datos Local: SQLite (mediante la librería Room de Android) para almacenamiento persistente en el dispositivo.
- Motor de Base de Datos Central: Relacional (PostgreSQL o MySQL) conectada vía SQLAlchemy en el entorno Flask.

3.1.4 Interfaces de comunicación

- Comunicación bajo el protocolo HTTP/HTTPS utilizando arquitectura REST. Los intercambios de datos se realizarán en formato JSON.
- Manejo de estados de red nativos de Android (ConnectivityManager) para disparar la sincronización.

3.2 Requisitos funcionales

3.2.1 Requisito funcional 1: Autenticación Segura

Actores: Técnico, Jefe, Gerencia, Administrador.

Descripción: El usuario debe poder ingresar sus credenciales (usuario/contraseña). El sistema validará los datos y retornará un token JWT, redirigiendo a la pantalla correspondiente a su rol.

3.2.2 Requisito funcional 2: Listado de Destinos (Caché Local)

Actores: Técnico de Servicio.

Descripción: El técnico debe poder desplegar una lista actualizada de clínicas/hospitales asignados. Esta lista debe cargarse de la base de datos local (Room) para estar disponible sin internet.

3.2.3 Requisito funcional 3: Captura de Odometría (Inicio y Fin)

Actores: Técnico de Servicio.

Descripción: El sistema debe obligar al técnico a ingresar numéricamente el valor del odómetro del vehículo al iniciar el viaje, y el valor del odómetro al llegar a la clínica, calculando la diferencia total de kilómetros.

3.2.4 Requisito funcional 4: Registro Fotográfico de Peajes

Actores: Técnico de Servicio.

Descripción: Si dependiendo de la ruta se identificaran los peajes por los que el técnico pasaria, el técnico marca la casilla "Pago de Peaje", el sistema desplegará un campo para el monto e invocará la cámara del teléfono para fotografiar el comprobante físico, comprimiendo la imagen internamente.

3.2.5 Requisito funcional 5: Almacenamiento y Encolamiento Offline

Actores: Sistema Móvil.

Descripción: Al presionar "Finalizar Viaje", si el dispositivo carece de red, el sistema guardará el registro completo localmente, generará un UUID (Identificador Único) y marcará el registro como "Pendiente de Sincronización".

3.2.6 Requisito funcional 6: Sincronización en Segundo Plano

Actores: Sistema Móvil / Backend.

Descripción: Al detectar conexión a internet, el sistema (vía WorkManager) enviará los viajes pendientes al servidor. El backend validará los UUIDs para descartar inserciones duplicadas.

3.2.7 Requisito funcional 7: Bandeja de Validaciones y Alertas

Actores: Jefe de Servicio Técnico.

Descripción: El jefe debe visualizar todos los viajes realizados. El sistema resaltará con una alerta visual en rojo aquellos viajes donde la distancia ingresada difiera (ej. > 10%) de la distancia oficial parametrizada para esa ruta.

3.2.8 Requisito funcional 8: Aprobación u Observación de Gastos

Actores: Jefe de Servicio Técnico.

Descripción: El jefe debe poder cambiar el estado de un viaje a "Aprobado" (listo para análisis financiero) o "Con Observaciones" (bloqueado, a la espera de justificación del técnico).

3.2.9 Requisito funcional 9: Dashboard de Costos y Rentabilidad

Actores: Gerencia / Finanzas.

Descripción: El sistema debe agrupar los viajes aprobados y mostrar el gasto de movilización total mensual y promedio por visita, con filtros por cliente (Clínica/Hospital) y rango de fechas.

3.2.10 Requisito funcional 10: Exportación de Reportes

Actores: Gerencia / Finanzas.

Descripción: El usuario debe poder presionar un botón para generar y descargar un archivo CSV con el detalle granular de todos los viajes asociados a un contrato para respaldar el cobro.

3.2.11 Requisito funcional 11: Mantenedor (ABM) de Parámetros

Actores: Administrador.

Descripción: El sistema debe proporcionar formularios web para crear, editar o dar de baja Vehículos (patente y rendimiento km/l), Clínicas (nombre y distancia oficial en km) y Usuarios.

3.2.12 Requisito funcional 12: Actualización de Tarifa de Combustible

Actores: Sistema / Administrador.

Descripción: El backend consultará una vez al día a una API pública para fijar el costo vigente por litro de combustible. El administrador debe disponer de un input manual para sobrecribir este valor en caso de caída de la API externa.

3.3 Requisitos no funcionales

3.3.1 Requisitos de rendimiento

- El registro de un viaje en la base de datos local del móvil (Room) debe demorar menos de 1 segundo para no interrumpir el flujo de trabajo del técnico.
- Las peticiones estándar de la interfaz web (Bandeja de Jefe) deben cargar la información en menos de 2 segundos.

3.3.2 Seguridad

- Toda petición HTTP hacia el backend (Flask) debe requerir validación mediante tokens JWT en la cabecera.
- Las contraseñas de todos los perfiles deben ser almacenadas utilizando encriptación unidireccional (ej. Bcrypt).
- El manejo de rutas y datos personales debe restringirse estrictamente por roles de usuario (RBAC).

3.3.3 Fiabilidad

- La aplicación móvil debe asegurar 0% de pérdida de datos transaccionales, operando de manera continua hasta 8 horas sin señal de internet.
- El mecanismo de UUID generado en cliente garantizará que fallos de red durante el envío no provoquen doble cobro o registros duplicados en la base de datos.

3.3.4 Disponibilidad

- El servicio backend, alojado en un entorno de producción (ej. Gunicorn/Nginx), deberá garantizar un uptime (disponibilidad) mínimo del 99.5% durante los horarios operativos.

3.3.5 Mantenibilidad

- El código móvil en Kotlin deberá estar estructurado rigurosamente bajo el patrón arquitectónico MVVM (Model-View-ViewModel).
- El código del backend en Python estructurará sus rutas bajo el patrón Blueprints de Flask para separar los módulos lógicos (Auth, Viajes, Reportes, Mantenedores).

3.3.6 Portabilidad

- La aplicación de terreno será compilada y deberá ser funcional en cualquier dispositivo con sistema operativo Android 8.0 (Oreo) o superior.
- La interfaz web administrativa debe ser 100% responsiva y compatible con navegadores modernos basados en Chromium y WebKit.

3.4 Otros Requisitos

- Peso de la aplicación: El archivo instalable generado para distribución a los técnicos (APK / App Bundle) no deberá superar los 30 MB para facilitar su descarga en terrenos con baja señal.